



38-MAVZU · IV BOB. PYTHON, POSTGRESQL VA DJANGO

CRUD amallarini yaratish. CRUD asosida mini loyiha

Maktab blogini to'liq CRUD ilovaga aylantirish: ro'yxat (sahifalash bilan), batafsil ko'rish, yaratish, tahrirlash, o'chirish (tasdiqlash bilan), faqat muallifga ruxsat; xuddi shu ishni Class Based View'lar (ListView, DetailView, CreateView, UpdateView, DeleteView) bilan qilish; sinovdan o'tkazish.

8-sinf · Web Full-stack dasturlash

Bugun nimani o'rganamiz?

- Model, View, Template va Form'ni **bitta to'liq ilovaga** birlashtirish
- 4 ta CRUD amali — veb-interfeys orqali
- **Ruxsatlar:** faqat muallif tahrirlaydi va o'chiradi
- **Sahifalash** (pagination)
- Xuddi shu ilova — tayyor **Class Based View**'lar bilan
- **Sinov** (21-loyiha: Blog veb-ilovasi)

Reja: CRUD = 5 ta sahifa

Amal	URL	Shablon	Kim uchun
R — ro'yxat	/	maqola_list.html	Hamma
R — batafsil	/maqola/<id>/	maqola_detail.html	Hamma
C — yaratish	/maqola/yangi/	maqola_form.html	Kirganlar
U — tahrirlash	/maqola/<id>/tahrirlash/	maqola_form.html (o'sha!)	Muallif
D — o'chirish	/maqola/<id>/ochirish/	maqola_confirm_delete.html	Muallif

Har qanday CRUD ilovaning "retsepti": kutubxona, talabalar, mahsulotlar...

1. URL'lar

blog/urls.py

```
urlpatterns = [  
    path("", views.maqola_royxat, name="bosh"),  
    path("maqola/yangi/", views.maqola_yaratish, name="yaratish"),    # <id> dan OLDIN  
    path("maqola/<int:id>/", views.maqola_batafsil, name="maqola"),  
    path("maqola/<int:id>/tahrirlash/", views.maqola_tahrirlash, name="tahrirlash"),  
    path("maqola/<int:id>/ochirish/", views.maqola_ochirish, name="ochirish"),  
]
```

- Aniq yo'l (`yangi/`) o'zgaruvchili yo'ldan **oldin** yoziladi

1. Forma

blog/forms.py

```
class MaqolaForm(forms.ModelForm):
    class Meta:
        model = Maqola
        fields = ["sarlavha", "kategoriya", "matn", "chop_etilgan"]

    def clean_sarlavha(self):
        s = self.cleaned_data["sarlavha"].strip()
        if len(s) < 5:
            raise forms.ValidationError("Sarlavha kamida 5 belgidan iborat bo'lsin")
        return s
```

2. Read: ro'yxat (sahifalash) va batafsil

Python

```
from django.core.paginator import Paginator

def maqola_royxat(request):
    maqolalar = Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya", "muallif")
    sahifa = Paginator(maqolalar, 6).get_page(request.GET.get("page")) # 6 tadan
    return render(request, "blog/maqola_list.html", {"sahifa": sahifa})

def maqola_batafsil(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    if not maqola.chop_etilgan and maqola.muallif != request.user:
        raise Http404 # qoralama - faqat muallifga
    return render(request, "blog/maqola_detail.html", {"maqola": maqola})
```

Django shablon

```
{% if sahifa.has_previous %}<a href="?page={{ sahifa.previous_page_number }}">< Oldingi</a>{% endif %}
{{ sahifa.number }} / {{ sahifa.paginator.num_pages }}
{% if sahifa.has_next %}<a href="?page={{ sahifa.next_page_number }}">Keyingi ></a>{% endif %}
```

3. Create va Update — bitta forma

Python

```
@login_required
def maqola_yaratish(request):
    form = MaqolaForm(request.POST or None)
    if request.method == "POST" and form.is_valid():
        maqola = form.save(commit=False)
        maqola.muallif = request.user
        maqola.save()
        messages.success(request, "Maqola chop etildi! 🎉")
        return redirect(maqola)
    return render(request, "blog/maqola_form.html", {"form": form, "sarlavha": "Yangi maqola"})

@login_required
def maqola_tahrirlash(request, id):
    maqola = get_object_or_404(Maqola, id=id)
    if maqola.muallif != request.user:
        return HttpResponseForbidden("Bu maqolani tahrirlash huquqingiz yo'q")
    form = MaqolaForm(request.POST or None, instance=maqola)  # ← instance!
    if request.method == "POST" and form.is_valid():
        form.save()
        return redirect(maqola)
    return render(request, "blog/maqola_form.html", {"form": form, "sarlavha": "Tahrirlash"})
```

4. Delete — tasdiqlash va POST

Python

```
@login_required
def maqola_ochirish(request, id):
    maqola = get_object_or_404(Maqola, id=id, muallif=request.user)
    if request.method == "POST":
        maqola.delete()
        messages.success(request, "Maqola o'chirildi")
        return redirect("blog:bosh")
    return render(request, "blog/maqola_confirm_delete.html", {"maqola": maqola})
```

maqola_confirm_delete.html

```
<h1>O'chirishni tasdiqlang</h1>
<p>«{{ maqola.sarlavha }}» maqolasini o'chirmoqchimisiz? Qaytarib bo'lmaydi.</p>
<form method="post">
    {% csrf_token %}
    <button class="tugma qizil">Ha, o'chirish</button>
    <a href="{{ maqola.get_absolute_url }}">Yo'q, ortga</a>
</form>
```


Xavfsizlik: ikki qoida

- 1 **Ruxsatni serverda tekshiring.** Tugmani shablonda yashirish yetmaydi — ayyor foydalanuvchi `/maqola/5/tahrirlash/` ni qo'lda yozadi
- 2 **O'chirish — faqat POST.** GET bilan o'chirsak, oddiy havola yoki begona saytdagi `<img>` maqolani o'chirib yuboradi

maqola_detail.html

```
{% if user == maqola.muallif %}           {# tugmalar faqat muallifga ko'rinadi #}  
  <a href="{% url 'blog:tahrirlash' maqola.id %}">✎ Tahrirlash</a>  
  <a href="{% url 'blog:ochirish' maqola.id %}">🗑 O'chirish</a>  
{% endif %}
```

Shablon — qulaylik uchun, view — xavfsizlik uchun

5. Class Based View'lar bilan

Python

```
from django.contrib.auth.mixins import LoginRequiredMixin, UserPassesTestMixin
from django.views.generic import ListView, DetailView, CreateView, UpdateView, DeleteView

class MaqolaList(ListView):
    queryset = Maqola.objects.filter(chop_etilgan=True)
    paginate_by = 6

class MaqolaDetail(DetailView):
    model = Maqola
    pk_url_kwarg = "id"
```

5. CBV: yaratish, tahrirlash, o'chirish

Python

```
class MaqolaCreate(LoginRequiredMixin, CreateView):
    model = Maqola
    form_class = MaqolaForm
    def form_valid(self, form):
        form.instance.muallif = self.request.user
        return super().form_valid(form)

class MuallifMixin(UserPassesTestMixin):
    def test_func(self):
        return self.get_object().muallif == self.request.user

class MaqolaUpdate(LoginRequiredMixin, MuallifMixin, UpdateView):
    model = Maqola
    form_class = MaqolaForm
    pk_url_kwarg = "id"

class MaqolaDelete(LoginRequiredMixin, MuallifMixin, DeleteView):
    model = Maqola
    pk_url_kwarg = "id"
    success_url = reverse_lazy("blog:bosh")
```

Tayyor CBV'lar

Klass	Nima qiladi
<code>ListView</code>	Ro'yxat + sahifalash
<code>DetailView</code>	Bitta obyekt (yo'q bo'lsa 404)
<code>CreateView</code>	Bo'sh forma → tekshirish → yaratish
<code>UpdateView</code>	Eski qiymatlar bilan forma → saqlash
<code>DeleteView</code>	Tasdiqlash + POST bilan o'chirish
<code>LoginRequiredMixin</code>	<code>@login_required</code> ning CBV varianti
<code>UserPassesTestMixin</code>	<code>test_func()</code> False → 403

- ~80 qator view → ~40 qator
- Shablon nomlari avtomatik: `model_list.html`, `model_detail.html`, `model_form.html`, `model_confirm_delete.html`

6. Sinovdan o'tkazish

#	Ssenariy	Kutilgan natija
1	Mehmon bosh sahifani ochadi	"Yangi maqola" tugmasi yo'q
2	Mehmon <code>/maqola/yangi/</code> ni ochadi	Login sahifasiga
3	Ali 3 belgili sarlavha bilan yaratadi	Xato, ma'lumot yo'qolmaydi
4	Ali to'g'ri maqola yaratadi	Maqola sahifasi + yashil xabar
5	Vali Ali'ning maqolasini ochadi	Tahrirlash tugmalari yo'q
6	Vali <code>/maqola/ID/tahrirlash/</code> ni qo'lda ochadi	403 Forbidden
7	Ali maqolani o'chiradi	Tasdiqlash → "Ha" → yo'q
8	15 ta maqola	3 ta sahifa
9	<code>/maqola/9999/</code>	404

- Ikki brauzerda (biri **Inkognito**) ikki foydalanuvchi bilan tekshiring

Avtomatik test

blog/tests.py

```
from django.test import TestCase

class MaqolaTestlari(TestCase):
    def setUp(self):
        self.ali = User.objects.create_user("ali", password="parol12345")
        self.vali = User.objects.create_user("vali", password="parol12345")
        kat = Kategoriya.objects.create(nomi="Sport", slug="sport")
        self.m = Maqola.objects.create(sarlavha="Test maqola", matn="...", kategoriya=kate, muallif=self.ali)

    def test_bosh_sahifa_ochiladi(self):
        self.assertEqual(self.client.get(reverse("blog:bosh")).status_code, 200)

    def test_begona_tahrirlay_olmaydi(self):
        self.client.login(username="vali", password="parol12345")
        javob = self.client.get(reverse("blog:tahrirlash", args=[self.m.id]))
        self.assertEqual(javob.status_code, 403)
```

Terminal

```
python manage.py test blog          # Ran 2 tests ... OK
```

- Testlar **vaqtinchalik bazada** ishlaydi — haqiqiy ma'lumotga tegmaydi

Amaliyot (21-loyiha)

- 1 URL, forma, view va shablonlarni qo'shing (funksiya variantida)
- 2 **9 ta ssenariyni** bajaring va belgilang
- 3 View'larni **CBV** variantiga o'tkazing va qayta tekshiring
- 4 "**Mening maqolalarim**" — `/mening/` (qoralamalar bilan)
- 5 GitHub'ga push; **README**: tavsif, texnologiyalar, ishga tushirish, skrinshotlar

Ko'p uchraydigan xatolar

- ❌ Aniq yo'llarni (`maqola/yangi/`) `<str:...>` dan **keyin** yozish
- ❌ Tahrirlashda `instance=` unutilgan — yangi nusxa yaratiladi
- ❌ Ruxsat **faqat shablonda** tekshirilgan
- ❌ **GET** so'rov bilan o'chirish
- ❌ `TemplateDoesNotExist: blog/maqola_form.html` — CBV standart nomni qidiriyapti

Savol

ModelForm bilan mavjud obyektни tahrirlash uchun formaga nima beriladi?

A `id=`

B `object=`

C `instance=`

D `update=True`

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

"Faqat muallif tahrirlay oladi" tekshiruvi qayerda bo'lishi SHART?

A View'da (serverda)

B Faqat shablonda

C CSS'da

D JavaScript'da

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

Ro'yxatni sahifalarga bo'luvchi Django klassi?

A Splitter

B PageView

C Limiter

D Paginator

Javobni tanlang — to'g'ri variant yashil bo'ladi

Savol

CBV'da tizimga kirishni talab qiluvchi mixin?

A AuthMixin

B UserPassesTestMixin

C LoginRequiredMixin

D SessionMixin

Javobni tanlang — to'g'ri variant yashil bo'ladi



IV bob yakunlovchi loyiha (bittasini tanlang)

- **Kutubxona boshqaruv tizimi** (22-loyiha): kitoblar CRUD, "olish/qaytarish"
- **Talabalar tizimi** (23-loyiha): talabalar, guruhlar, baholar, o'rtacha
- **E'lonlar taxtasi**: rasm bilan e'lonlar, toifa va qidiruv filtri
- Talab: model + admin + forma + 5 sahifa + ruxsatlar + sahifalash + 2 ta test

IV bob yakunlandi!

- CRUD = ro'yxat, batafsil, yaratish, tahrirlash, o'chirish
- `ModelForm(request.POST, instance=obj)` — bitta forma
- Ruxsatlar **serverda**; o'chirish — tasdiqlash + POST
- `Paginator`; `ListView`, `DetailView`, `CreateView`, `UpdateView`, `DeleteView`

Keyingi bob: API qo'shamiz va saytni internetga chiqaramiz!



Savollar bormi?

Keyingi mavzu: **39. Django REST Framework (DRF) ga kirish. Serializerlar va API View'lar**

Batafsil matn, misollar va testlar: **38-django-crud-loyiha.html** darsida